OpenZeppelin | security

Predicate Contracts Audit

PREDICATE

November 11, 2025

Table of Contents

Table of Contents 2

Summary 3

Scope 4

System Overview 5

Attested Authorization Flow 5

Security Model and Trust Assumptions 6

Privileged Roles 6

Low Severity 7

L-01 Floating Pragma 7

L-02 Incomplete Docstrings 7

L-03 Registry Switch Causes Policy Desynchronization and Attestation Failure 7

L-04 Attester Enumeration Could Support Pagination 8

L-05 Redundant Event Emissions in Setters 9

L-06 Missing Address Validation in Attester Registration 9

L-07 Missing Docstrings 10

L-08 Missing Domain Separation Enables Cross-Chain and Cross-Registry Replay of Attestations 10

Notes & Additional Information 11

N-01 Use calldata Instead of memory 11

N-02 Missing Security Contact 11

N-03 Non-Explicit Imports 12

N-04 Unprotected Proxy Initialization Enables Ownership Hijacking 12

N-05 Use Custom Errors 13

N-06 Outdated Solidity Versions 13

N-07 Redundant Getter Function 14

N-08 Policy Identifier Format Not Enforced 14

N-09 Unnecessary Data Field in Event Emission 15

N-10 Unbounded Loop in Attester Deregistration Creates Operational Risk 15

N-11 Deregistration Immediately Revokes Unexpired Attestations 15

Conclusion 17

Appendix 18

Issue Classification 18

Summary

Type	DeFi
Timeline	From 2025-11-11 To 2025-11-12
Languages	Solidity

Total Issues	19 (13 resolved)
Critical Severity Issues	0 (0 resolved)
High Severity Issues	0 (0 resolved)
Medium Severity Issues	0 (0 resolved)
Low Severity Issues	8 (6 resolved)
Notes & Additional Information	11 (7 resolved)

Scope

OpenZeppelin performed an audit of the predicatelabs/predicate-contracts repository at commit 2c8a126.

In scope were the following files:

```
src
├── PredicateRegistry.sol
└── mixins
    └── PredicateClient.sol
```

System Overview

The attested-call architecture of Predicate splits responsibilities between a shared registry and application-specific client contracts. `PredicateRegistry` curates the list of trusted attesters, stores every client's policy identifier, and validates attestations by reconstructing a `Statement`, enforcing expiry and single-use UUIDs, and recovering the ECDSA signer before emitting `StatementValidated`.

`PredicateClient` is a reusable mixin that downstream contracts inherit to keep the registry address and policy inside ERC-7201 namespaced storage, push policy updates on-chain, and assemble the `Statement` payload sent for authorization. This separation lets business-logic contracts remain small while delegating signature verification, replay defenses, and attester governance to one upgradeable component.

Attested Authorization Flow

The attested authorization flow is as follows:

1. A client contract calls `_initPredicateClient` to store the registry address and publish its policy identifier.
2. Before executing protected logic, the client packages the internal function signature, arguments, caller, and value into a `Statement`, pairs it with an attester-provided `Attestation`, and forwards both to the registry.
3. `validateAttestation` replaces the declared target with `msg.sender` inside `hashStatementSafe` (preventing cross-contract replays), checks expiration and UUID freshness, recovers the signer, and verifies the attester is still registered.
4. Once validation succeeds and the UUID is marked as used, the client proceeds, ensuring a registered attester approved that exact invocation context.

Security Model and Trust Assumptions

The design assumes that the registry owner maintains an accurate attester set and that attesters protect their keys while signing statements that faithfully encode the intended calldata and policy. Statement hashing relies on plain `keccak256` without an explicit domain separator, so deployers trust that binding `msg.sender` in `hashStatementSafe` adequately prevents cross-chain or multi-environment replay. Although, shared keys across chains may still need stronger domain separation. Policies are advisory: the registry does not compare a statement's policy to the latest stored value, so clients must keep policies up to date and attesters must refuse stale identifiers. UUIDs provide one-time replay protection, meaning any desynchronization between clients and the registry can permanently invalidate pending attestations.

Privileged Roles

The `PredicateRegistry` owner (an `Ownable2StepUpgradeable` admin) exclusively controls `registerAttester` and `deregisterAttester`, directly defining who can authorize downstream transactions. Contracts inheriting `PredicateClient` typically wrap `_setRegistry`, `_setPolicyID`, or `_initPredicateClient` inside project-specific admin functions. Compromising these roles enables registry swaps, policy downgrades, or denial of service. Registered attesters themselves are quasi-privileged because any key compromise can authorize arbitrary client calls until the owner removes the attester or the existing attestations expire.

Low Severity

L-01 Floating Pragma

Pragma directives should be fixed to clearly identify the Solidity version with which the contracts will be compiled.

- `PredicateRegistry.sol` has the `solidity ^0.8.12` floating pragma directive.
- `PredicateClient.sol` has the `solidity ^0.8.12` floating pragma directive.

Consider using fixed pragma directives.

Update: *Resolved within pull request 38 at commit 5db55d5.*

L-02 Incomplete Docstrings

Within `PredicateClient.sol`, multiple instances of incomplete docstrings were identified:

- The `PredicateRegistryUpdated` event, the `oldRegistry`, `newRegistry` parameters are not documented.
- The `PredicatePolicyIDUpdated` event, the `oldPolicyID`, `newPolicyID` parameters are not documented.

Consider thoroughly documenting all functions/events (and their parameters or return values) that are part of a contract's public API. When writing docstrings, consider following the Ethereum Natural Specification Format (NatSpec).

Update: *Resolved within pull request 38 at commit 5db55d5.*

L-03 Registry Switch Causes Policy Desynchronization and Attestation Failure

The `PredicateClient` contract maintains a local policy cache in ERC-7201 namespaced storage along with a reference to the `PredicateRegistry` contract used for attestation validation. When `_setPolicyID` updates the policy, it modifies both the local cache and registers the policy with the current registry. However, the `_setRegistry` function replaces

the registry reference without re-registering the cached policy in the new registry, creating a desynchronization where the client's cached policy differs from what the new registry has recorded.

This mismatch causes complete service disruption because off-chain attesters query the new registry's `getPolicyID(address)` to determine which policy to sign and receive an empty string since the client was never registered, while the client continues building statements with its cached policy during transaction validation. This results in hash mismatches that cause signature recovery to fail and all attestations to revert with "Invalid signature" until an administrator manually re-registers the policy.

Consider automatically re-registering the cached policy when `_setRegistry` is called by adding `$.registry.setPolicyID($.policy)` after updating the registry reference. Alternatively, consider modifying `getPolicyID` to always read from the current registry instead of returning the cached value. Doing so would help ensure that off-chain and on-chain systems remain synchronized during registry transitions.

Update: *Resolved within pull request 40 at commit 460c116.*

L-04 Attester Enumeration Could Support Pagination

The `getRegisteredAttesters` function returns all registered attesters in a single call. This function is marked as `view` and incurs no gas cost when called externally (as intended). However, integrators who call it from state-modifying functions would copy the entire array to memory, with gas costs scaling linearly with the number of registered attesters.

Consider adding pagination support through offset and limit parameters to enable efficient retrieval of attester subsets. This would benefit integrators building on-chain systems that need to enumerate attesters within transaction contexts while reducing the risk of unexpected gas consumption patterns.

Update: *Acknowledged, Not Resolved. The team stated:*

> *We've decided to defer this suggestion at this time.*

L-05 Redundant Event Emissions in Setters

The `_setPolicyID` and `_setRegistry` functions update configuration parameters in the `PredicateClient` contract and emit events to notify off-chain systems of the changes. Both functions emit events even when the new value equals the old value, wasting gas on unnecessary storage operations and event emissions. This can potentially confuse off-chain indexers that track state changes with no-op updates that do not actually modify contract state.

Consider adding equality checks before performing state updates and emitting events, ensuring that changes only occur and are announced when the new value differs from the existing one.

Update: *Resolved within pull request 38 at commit 5db55d5.*

L-06 Missing Address Validation in Attester Registration

The `registerAttester` function allows the contract owner to add new attesters who are authorized to sign compliance attestations for transaction authorization. However, the function accepts `address(0)` and contract addresses without validation, which can create operational issues since `ECDSA.recover` can only return externally owned account addresses and contract addresses cannot produce valid ECDSA signatures, resulting in unusable attester slots that waste storage when accidentally registered. In addition, removing these invalid entries requires the `O(n)` deregistration process, making cleanup increasingly expensive as more attesters are registered.

Consider adding input validation to reject `address(0)` and verifying that the provided address is an EOA by checking that `code.length == 0`.

Update: *Acknowledged, Not Resolved. The team stated:*

> *We've decided to defer this suggestion at this time.*

L-07 Missing Docstrings

Throughout the codebase, multiple instances of missing docstrings were identified:

- In `PredicateRegistry.sol`, the `registeredAttesters`, `isAttesterRegistered`, `clientToPolicy`, `usedStatementUUIDs` state variables and the `AttesterRegistered`, `AttesterDeregistered`, `PolicySet`, `StatementValidated` events
- In `PredicateClient.sol`, the `getPolicyID`, `getRegistry` functions

Consider thoroughly documenting all functions (and their parameters) that are part of any contract's public API. Functions implementing sensitive functionality, even if not public, should be clearly documented as well. When writing docstrings, consider following the Ethereum Natural Specification Format (NatSpec).

Update: *Resolved within pull request 38 at commit 5db55d5.*

L-08 Missing Domain Separation Enables Cross-Chain and Cross-Registry Replay of Attestations

The `PredicateRegistry` contract constructs the message digest in `hashStatementWithExpiry` and `hashStatementSafe` functions, without including chain- or contract-level domain data. The `validateAttestation` function then verifies signatures against this digest and marks `usedStatementUUIDs[_statement.uuid]` only within the local deployment. While `hashStatementSafe` replaces `_statement.target` with `msg.sender` to bind the signature to the calling contract, neither function binds the digest to `block.chainid` or the registry's `address(this)`.

As a result, an attestation signed off-chain over `hashStatementWithExpiry` can be replayed on another chain or another registry instance if the caller contract address is reproduced (e.g., via CREATE2) and the attester is registered there. In the current implementation, where attestations authorize the spender rather than the receiver, this does not create an immediate security issue. However, any integrator implementing patterns where the validated address differs from the transaction initiator (such as `transferFrom`, meta-transactions, or permit-style functions) will face cross-chain replay vulnerabilities.

Consider introducing explicit domain separation in both `hashStatementWithExpiry` and `hashStatementSafe` by including `block.chainid` and the registry `address(this)` in the encoded data, while retaining `msg.sender` to bind to the intended caller. Alternatively, consider migrating to EIP-712 typed data with a domain separator that includes `name`,

`version`, `chainId`, and `verifyingContract`, and ensuring that off-chain signers produce signatures over the exact digest used by `validateAttestation`.

Update: *Resolved within pull request 37 at commit 934bb48.*

Notes & Additional Information

N-01 Use `calldata` Instead of `memory`

When dealing with the parameters of `external` functions, it is more gas-efficient to read their arguments directly from `calldata` instead of storing them to `memory`. `calldata` is a read-only region of memory that contains the arguments of incoming `external` function calls. This makes using `calldata` as the data location for such parameters cheaper and more efficient compared to `memory`. Thus, using `calldata` in such situations will generally save gas and improve the performance of a smart contract.

Within `PredicateRegistry.sol`, the `_policyID` parameter should use `calldata` instead of `memory`.

Consider using `calldata` as the data location for the parameters of `external` functions to optimize gas usage.

Update: *Resolved within pull request 39 at commit 34cf1e6.*

N-02 Missing Security Contact

Providing a specific security contact (such as an email address or ENS name) within a smart contract significantly simplifies the process for individuals to communicate if they identify a vulnerability in the code. This practice is quite beneficial as it permits the code owners to dictate the communication channel for vulnerability disclosure, eliminating the risk of miscommunication or failure to report due to a lack of knowledge on how to do so. In addition, if the contract incorporates third-party libraries and a bug surfaces in those, it becomes easier for their maintainers to contact the appropriate person about the problem and provide mitigation instructions.

Consider adding a NatSpec comment containing a security contact above each contract definition. Using the `@custom:security-contact` convention is recommended as it has been adopted by the OpenZeppelin Wizard and the ethereum-lists.

Update: *Resolved within pull request 39 at commit 34cf1e6.*

N-03 Non-Explicit Imports

The use of non-explicit imports in the codebase can decrease code clarity and may create naming conflicts between locally-defined and imported variables. This is particularly relevant when multiple contracts exist within the same Solidity file or when inheritance chains are long.

Within `PredicateClient.sol`, the import of `IPredicateClient.sol` is a global import.

Following the principle that clearer code is better code, consider using the named import syntax (`import {A, B, C} from "X"`) to explicitly declare which contracts are being imported.

Update: *Resolved within pull request 39 at commit 34cf1e6.*

N-04 Unprotected Proxy Initialization Enables Ownership Hijacking

The `PredicateRegistry` contract is upgradeable and relies on an external `initialize(address _owner)` function guarded only by the `initializer` modifier. The initializer flow invokes `__Ownable2Step_init()` and then sets ownership via `__Ownable_init(_owner)`. No caller restriction is applied to `initialize`, and the implementation contract does not disable initializers in a constructor.

If an ERC-1967 proxy pointing to P`redicateRegistry` is deployed without atomic initialization, the first external caller can invoke `initialize(attacker)` and become the owner. The initializer modifier then locks the initializer, preventing remediation through re-initialization. The seized ownership enables full control of privileged functions such as `registerAttester` and `deregisterAttester`, impacting the integrity of the attester registry. Additionally, the absence of a constructor calling `_disableInitializers()` leaves the implementation contract itself initializable, which, while not compromising the proxy directly, is a best-practice gap and can enable misuse of the implementation instance.

Consider enforcing atomic initialization at deployment time by supplying initialization calldata to the proxy constructor or initializing in the same transaction via a trusted factory/deployer. Consider also adding a constructor to the implementation that calls `_disableInitializers()` to prevent initialization of the implementation contract. In addition, consider using established deployment tooling (e.g., OpenZeppelin Transparent/ UUPS helpers) which ensures that initialization occurs atomically and reduces the risk of uninitialized proxies.

Update: *Acknowledged, Not Resolved. The team stated:*

> *Our current and future deployment patterns protect against this by checking the ownership of the contract before using it in production.*

N-05 Use Custom Errors

Since Solidity version `0.8.4`, custom errors provide a cleaner and more cost-efficient way to explain to users why an operation failed.

Multiple instances of `revert` and/or `require` messages were found within `PredicateRegistry.sol`.

For conciseness and gas savings, consider replacing `require` and `revert` messages with custom errors.

Update: *Acknowledged, Not Resolved. The team stated:*

> *We've decided to defer this suggestion at this time.*

N-06 Outdated Solidity Versions

Using an outdated version of Solidity can lead to vulnerabilities and unexpected behavior in contracts. As such, it is important to keep the Solidity compiler version up to date.

Both `PredicateRegistry.sol` and `PredicateClient.sol` are using an outdated Solidity version (`^0.8.12`).

Consider taking advantage of the latest Solidity version to improve the overall readability and security of the codebase. Regardless of which version of Solidity is used, consider pinning the version consistently throughout the codebase to prevent bugs due to incompatible future releases.

Update: *Resolved within pull request 38 at commit 5db55d5.*

N-07 Redundant Getter Function

When state variables use `public` visibility in a contract, a getter method for the variable is automatically included.

Within the `PredicateRegistry` contract, the `getPolicyID` function is redundant because the `clientToPolicy` state variable already has a getter.

To improve the overall clarity, intent, and readability of the codebase, consider removing any redundant getter functions.

Update: *Acknowledged, Not Resolved. The team stated:*

> *The Predicate Registry implements an `IPredicateRegistry` interface which was not included in the audit. This is used by our front-end application and as such we felt that this change would be more disruptive than helpful at this point.*

N-08 Policy Identifier Format Not Enforced

The `setPolicyID` function accepts arbitrary strings without validation, including empty strings and formats that do not match the documented convention of "x-{hash[:16]}" for hash-based identifiers.

While the documentation explicitly states that format validation is not performed and various identifier types are supported including IPFS CIDs and URLs, the lack of constraints creates ambiguity about whether empty strings represent intentional opt-out behavior, unset policies, or configuration errors.

Consider documenting the intended semantics of empty policy strings and optionally adding validation to reject empty values if they represent misconfigurations. Alternatively, consider clarifying that empty strings are valid and describe their intended meaning in the system's policy model.

Update: *Acknowledged, Not Resolved. The team stated:*

> *We've decided to defer this suggestion at this time.*

N-09 Unnecessary Data Field in Event Emission

In `PredicateRegistry.sol`, the `emit PolicySet(msg.sender, msg.sender, _policyID, block.timestamp);` event emission includes unnecessary data fields such as `block.number` or `block.timestamp` as well as a duplication of the identical fields `setter` address and `client` address.

To improve the efficiency of the contract, consider removing unnecessary data fields from the event emission such as `block.number` or `block.timestamp` and deduplicate `setter` and `client` address.

Update: *Resolved within pull request 39 at commit 34cf1e6.*

N-10 Unbounded Loop in Attester Deregistration Creates Operational Risk

The `deregisterAttester` function is responsible for removing attesters from the registry. This may be necessary during security incidents such as private key compromise or when rotating attesters for operational reasons. The function iterates through the entire `registeredAttesters` array using an `O(n)` loop to locate and remove the target attester. This causes gas costs to scale linearly with the number of registered attesters and potentially hitting block gas limits when the registry contains hundreds or thousands of attesters, thereby preventing the urgent removal of compromised attesters during critical security incidents.

This operational limitation is particularly severe when combined with the attester deregistration issue, as administrators may be unable to remove malicious attesters while simultaneously being unable to allow existing valid attestations to continue functioning.

Consider refactoring the deregistration mechanism to use `O(1)` removal by storing each attester's array index in a mapping, enabling constant-time lookups and removals regardless of the total number of registered attesters in the system.

Update: *Resolved within pull request 36 at commit 4960df4.*

N-11 Deregistration Immediately Revokes Unexpired Attestations

The contract documentation for `deregisterAttester` states that “Existing attestations from this attester remain valid until their expiration". However, attestations are signed off-chain

over `hashStatementWithExpiry` or `hashStatementSafe`, which do not include any snapshot of the attester's registration state at the time of signing. During validation, `validateAttestation` recovers the signer and performs registration checks to authorize the use of an attestation.

In contrast, the implementation enforces registration at validation time by requiring `isAttesterRegistered[recoveredAttester]` to be `true`. When `deregisterAttester` sets `isAttesterRegistered[_attester] = false`, otherwise valid and unexpired attestations from that attester begin to revert in `validateAttestation`. Since the signed payload lacks any registration epoch, nonce, or issuance timestamp, there is no way to prove that the attester was registered at signing time, causing a direct mismatch with the stated guarantee and inconsistent behavior if the attester is later re-registered.

Consider aligning code and specification. If preserving previously issued attestations is intended, include a registration snapshot in the signed payload (e.g., an `attesterEpoch`/ `registrationNonce` or an `issuedAt` timestamp) and track per-attester disablement state (e.g., `disableFrom` timestamp or epoch). Validation should accept attestations where the signed snapshot indicates issuance during a registered period and `block.timestamp <= expiration`. Alternatively, consider maintaining a per-attester `validUntil` cutoff instead of flipping `isAttesterRegistered` immediately, or revise the documentation to state that deregistration revokes all attestations immediately.

Update: *Resolved within pull request 35 at commit 2af2531. The team stated:*

> *Clarified docs to correctly explain behavior.*

Conclusion

The Predicate Contracts v2 system provides a simplified, production-ready implementation for on-chain compliance verification through attestation-based validation. It features a central `PredicateRegistry` contract for managing attesters and validating cryptographic attestations alongside a `PredicateClient` mixin that enables seamless integration into smart contracts requiring policy enforcement.

The audit identified one high-severity issue concerning cross-chain replay attacks due to missing chain ID validation, along with three medium-severity issues pertaining to attester deregistration breaking valid attestations, policy update race conditions, and unbounded deregistration loops that could prevent urgent security responses. In addition, several low-severity issues and informational notes were also reported to improve code readability and facilitate future audits, integrations, and development.

The Predicate team is appreciated for being responsive and collaborative throughout the audit process, providing clear documentation and a well-structured codebase, which were essential to understanding the system.

Appendix

Issue Classification

OpenZeppelin classifies smart contract vulnerabilities on a 5-level scale:

- Critical
- High
- Medium
- Low
- Note/Information

Critical Severity

This classification is applied when the issue's impact is catastrophic, threatening extensive damage to the client's reputation and/or causing severe financial loss to the client or users. The likelihood of exploitation can be high, warranting a swift response. Critical issues typically involve significant risks such as the permanent loss or locking of a large volume of users' sensitive assets or the failure of core system functionalities without viable mitigations. These issues demand immediate attention due to their potential to compromise system integrity or user trust significantly.

High Severity

These issues are characterized by the potential to substantially impact the client's reputation and/or result in considerable financial losses. The likelihood of exploitation is significant, warranting a swift response. Such issues might include temporary loss or locking of a significant number of users' sensitive assets or disruptions to critical system functionalities, albeit with potential, yet limited, mitigations available. The emphasis is on the significant but not always catastrophic effects on system operation or asset security, necessitating prompt and effective remediation.

Medium Severity

Issues classified as being of medium severity can lead to a noticeable negative impact on the client's reputation and/or moderate financial losses. Such issues, if left unattended, have a moderate likelihood of being exploited or may cause unwanted side effects in the system.

These issues are typically confined to a smaller subset of users' sensitive assets or might involve deviations from the specified system design that, while not directly financial in nature, compromise system integrity or user experience. The focus here is on issues that pose a real but contained risk, warranting timely attention to prevent escalation.

Low Severity

Low-severity issues are those that have a low impact on the client's operations and/or reputation. These issues may represent minor risks or inefficiencies to the client's specific business model. They are identified as areas for improvement that, while not urgent, could enhance the security and quality of the codebase if addressed.

Notes & Additional Information Severity

This category is reserved for issues that, despite having a minimal impact, are still important to resolve. Addressing these issues contributes to the overall security posture and code quality improvement but does not require immediate action. It reflects a commitment to maintaining high standards and continuous improvement, even in areas that do not pose immediate risks.